Machine Learning in Engineering Science

Week 17: Project integration and oral-exam preparation

Chi-Hua Jonny Yu / NCKU + LAiMM

Fall 2026 / 2026-12-28

Chapter 1

Project integration and oral-exam preparation

Week 17: 2026-12-28, Monday 14:00–17:00.

Package the same synthetic workflow in readable source and JSON. Verify exact payload bytes, run a new Python process from a new working directory, and distinguish numerical replay from physical or model validation.

Learning outcome

Package a bounded workflow, reproduce it in a fresh process, audit artifacts, and defend claim, evidence and limitation.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

1.1 H1: Integrate the final evidence contract

1.1.1 Integration means a reconstructable result

Connect input, source, code, configuration and output into one evidence chain.

Worked reasoning. Artifact identities; manifest; runner; environment; replay comparison; oral explanation.

Check. What does a project screenshot omit?

Explanation and boundary. Executable procedure, inputs, dependency versions, rejected cases and the basis of its claims.

1.1.2 Scope the final claim before packaging

State exactly what the synthetic workflow demonstrated and did not demonstrate.

Worked reasoning. The frozen local implementation matches these independent teaching fixtures; no physical deployment or live-model evaluation is established.

Check. Why not say the agent is reliable?

Explanation and boundary. That omits the task, tested cases, assumptions and scope. Use a claim that the exported evidence can actually support.

1.1.3 The manifest names required payloads

Hash input, protocol, configuration and source; exclude the manifest from hashing itself.

Worked reasoning. A file cannot generally include its own final hash. Keep the required payload hashes in a separate manifest.

Check. Does a matching unsigned manifest authenticate a sender?

Explanation and boundary. No. Someone can replace both payload and digest. Trust needs an external reference or signature.

1.1.4 Use canonical JSON deliberately

Sort keys and fixed serialization identify one representation, not all equivalent objects.

Worked reasoning. Whitespace changes a raw-file hash. Canonical hashing compares the declared serialized representation.

Check. Which hash should the handover record use?

Explanation and boundary. Declare whether it hashes exact files or canonical values. This package uses exact saved payload bytes for integrity.

1.1.5 Source is safer to inspect than opaque objects

A readable runner rebuilds the workflow from supplied files.

Worked reasoning. The package contains JSON and Python source, not serialized executable model objects.

Check. Does readable source make execution safe?

Explanation and boundary. No. Review it and use an appropriate environment. Arbitrary Python can still perform harmful operations.

1.1.6 A new process removes hidden globals

Run the saved source from a fresh working directory with explicit paths.

Worked reasoning. The runner must not require an in-memory variable, previous cell or current notebook directory.

Check. What does a fresh process still share?

Explanation and boundary. It can share interpreter, packages, operating system and hardware. It is not independent scientific replication.

1.1.7 Replay compares computations, not outcomes

A zero replay difference does not imply an accurate physical model.

Worked reasoning. Prediction error compares against observations; replay difference compares outputs of the same declared procedure.

Check. Can an incorrect workflow replay perfectly?

Explanation and boundary. Yes. Reproducibility preserves the computation, including its defects. Independent fixtures test correctness separately.

1.1.8 Align identities before values

Compare case IDs and output fields before applying numeric tolerance.

Worked reasoning. A comparison must join by stable IDs or require the declared sequence. Bare positional vectors can mismatch cases.

Check. Does matching aggregate mean prove row alignment?

Explanation and boundary. No. Permutations can preserve an aggregate while assigning values to the wrong cases.

1.1.9 Record the actual environment

Save Python, package versions, platform, source hash and invocation.

Worked reasoning. The replay runner is standard-library only; plotting stays in the notebook. Preserve both scopes.

Check. Does a seed guarantee cross-version equality?

Explanation and boundary. No. Algorithms, libraries and numeric implementations can change. Name the environment and comparison criterion.

1.1.10 Summary: package the claim with its limits

An honest handover includes failures, replay evidence and missing validation.

Worked reasoning. A different protocol, unit convention or population can invalidate the original conclusion.

Check. What will the next hour test?

Explanation and boundary. Extracted files, corrupted payloads, explicit failure messages and a complete five-minute explanation.

1.2 H2: Replay, integrity checks, and oral defense

1.2.1 The ZIP must contain everything it promises

Use a required-file list rather than archiving arbitrary runtime state.

Worked reasoning. Read source and paths. The reference extraction rejects unexpected members and does not execute received code automatically.

Check. Why avoid arbitrary runtime archives?

Explanation and boundary. They may leak secrets, personal files or irrelevant state and make the intended procedure unclear.

1.2.2 Task 4: extract into a fresh directory

Verify manifest keys and bytes before starting the runner.

Worked reasoning. Removing protocol.json must fail before the analysis claims a protocol-based result.

Check. What should a failed integrity check do?

Explanation and boundary. Stop with a specific error. Do not retrain, repair or regenerate missing evidence silently.

1.2.3 Task 5: replay from saved files

Use an explicit interpreter, runner path, bundle root and output path.

Worked reasoning. A launched process is not completed execution. Check successful termination and parse its result file.

Check. Why preserve stderr on failure?

Explanation and boundary. It is evidence of the failing dependency or predicate. Replacing it with a success message destroys diagnosability.

1.2.4 Task 6: compare an aligned result vector

Use case IDs then the declared absolute-plus-relative tolerance.

Worked reasoning. This small runner should agree numerically in this environment. Report the actual maximum difference, not a promised zero.

Check. What if a metric agrees but predictions differ?

Explanation and boundary. Do not call it a successful replay. Aggregate agreement can conceal per-case differences.

1.2.5 Tamper detection is not authenticity

A changed copied payload should fail against the preserved manifest.

Worked reasoning. A harmless-looking blank line changes the exact-file hash even if a parser would ignore it.

Check. What if both payload and manifest are replaced?

Explanation and boundary. Unsigned consistency checks may pass. A trusted external reference or signature is needed for sender authenticity.

1.2.6 Explain the mechanism in a short defense

Use claim, evidence, limitation and an invalidating change.

Worked reasoning. Name the fixture, show the trace, recompute a value and identify what the evidence does not establish.

Check. Why is memorizing a score insufficient?

Explanation and boundary. It does not demonstrate the split, procedure, source or reasoning that gives the score meaning.

1.2.7 Budget the five-minute demonstration

Make every segment point to a real artifact.

Worked reasoning. Show one exact input, one executed success, one expected rejection, then how a peer can replay it.

Check. Must every student submit a new video this week?

Explanation and boundary. No. This is in-class rehearsal only; the separately announced stock-prediction HW retains its own five-minute video requirement.

1.2.8 Written and oral assessment remain separate

The project evidence supports explanation but does not replace either examination.

Worked reasoning. What breaks if a threshold changes units, a tool gains side effects or a retrieved source loses a qualifier?

Check. Should the course publish live exam answers here?

Explanation and boundary. No. These are public study prompts and worked practice, not confidential examination papers or new grading rules.

1.2.9 Peer handover is an adversarial usability test

A partner locates inputs, expected behavior and one known failure without coaching.

Worked reasoning. Capture what the partner could and could not reconstruct. Incomplete handover remains a documented limitation.

Check. Can a partner's successful replay establish generalization?

Explanation and boundary. No. It supports portability within the checked environment, not external predictive or security validity.

1.2.10 Summary: preserve the observed state

Keep passed, failed and untested items distinct in the final result card.

Worked reasoning. Save source, inputs, manifest, replay result and draft/completed response record.

Check. What must not be upgraded into completion?

Explanation and boundary. A prepared ZIP, launched job or connected runtime without successful execution and inspected outputs.

1.3 H3: Final integration studio and explain-back

1.3.1 Finish a rerunnable bounded project record

This lab integrates existing evidence rather than adding untested autonomy.

Worked reasoning. Restore 5 min; package and replay 20 min; integrity challenge 15 min; peer defense and export 20 min.

Check. What counts as an honest stopping point?

Explanation and boundary. A draft with explicit missing evidence and next action, not fabricated success or copied metrics.

1.3.2 Task 1: locate every hidden dependency

List expected payloads and identify any notebook-only variable.

Worked reasoning. Use a fresh working directory; the runner should use explicit paths rather than assume the notebook location.

Check. What if it only runs beside the notebook?

Explanation and boundary. The portability claim is too broad. Fix the path contract or state the required working directory explicitly.

1.3.3 Task 2: check case identity and ordering

Reorder a copied output and detect sequence mismatch before numeric comparison.

Worked reasoning. A length match alone does not establish observation identity.

Check. When is an explicit ID join acceptable?

Explanation and boundary. When IDs are unique and required membership is checked, and the alignment rule is recorded in the evidence.

1.3.4 Task 3: state the actual environment

Read the generated version record rather than an assumed Colab configuration.

Worked reasoning. A Colab link is a launch route, not proof that the notebook was run in a Google Colab browser.

Check. Why does this distinction matter?

Explanation and boundary. Dependency, network and runtime behavior can differ. Execution claims should name the environment actually tested.

1.3.5 Task 4: inspect the extracted package

Check file whitelist, manifest completeness and payload hashes.

Worked reasoning. Do not replace an absent file with a freshly invented one and call the same bundle complete.

Check. What does an intact archive prove?

Explanation and boundary. Its stored bytes can be read without CRC errors; it does not prove scientific correctness or code safety.

1.3.6 Task 5: require actual process completion

Check return code and parsed output after the new interpreter exits.

Worked reasoning. A failed or timed-out process remains an observed failure with its diagnostic output.

Check. What is the acceptance condition?

Explanation and boundary. Successful termination, required result schema and the declared per-case agreement checks.

1.3.7 Task 6: explain computational agreement

Compare saved and replayed means by case ID under fixed tolerance.

Worked reasoning. If nonzero but within tolerance, report that fact rather than replacing it with zero.

Check. What does zero difference leave untested?

Explanation and boundary. Physical data validity, external populations, real LLM behavior and unseen adversarial inputs.

1.3.8 Task 7: detect copied-file tampering

Modify a temporary payload and preserve the original reference.

Worked reasoning. A mismatch must prevent analysis from proceeding under the reference claim.

Check. Can a matching digest replace peer source review?

Explanation and boundary. No. Integrity and trust are distinct; a harmful file can have a perfectly matching digest.

1.3.9 Task 8: defend claim and limitation

A partner asks one unexpected question about mechanism or scope.

Worked reasoning. What if the readings are correlated or the protocol changes? Separate computational validity from a changed scientific task.

Check. What is a strong answer?

Explanation and boundary. One grounded in an actual artifact and explicit assumptions, with no invented deployment or accuracy claims.

1.3.10 Summary: prepare, do not replace, the final exam

Optional A tests metadata-only changes; optional B expands a handover checklist.

Worked reasoning. State passed, failed, untested and next action. Week 18 remains the written and oral final examination.

Check. What should continue after this course?

Explanation and boundary. The habit of defining the task, preserving data, checking execution, explaining results and bounding every claim.

1.4 Replay reference

$$\Delta_i = \text{replayed mean}_i - \text{reference mean}_i.$$

Compare unique case identities and declared order before values. Use the saved absolute-plus-relative tolerance. A zero maximum replay difference establishes agreement between computations in the checked environment, not physical accuracy, scientific replication or live-model quality.

1.5 Eight core practice tasks

1.5.1 Task 1: Find notebook-only dependencies

Predict which globals a new process can see. Inspect the saved source and explicit path arguments before executing.

Self-check. A new interpreter still shares the installed Python environment and operating system. Do not call it independent scientific replication.

1.5.2 Task 2: Check identity before numeric values

Reverse a copied result list, detect a sequence mismatch, then align explicitly by unique stable IDs.

Self-check. A length match or aggregate score is not an ID alignment check. Explicitly reject missing or duplicate IDs.

1.5.3 Task 3: Record the environment that actually ran

Inspect the actual Python and package record. A Colab launch URL does not establish Colab-browser execution.

Self-check. If you run in Colab, record its actual versions. Cross-version bitwise identity is not promised.

1.5.4 Task 4: Extract a whitelisted handover ZIP

Inspect each archive member and its manifest before execution. The supplied extractor accepts exactly six known base names.

Self-check. A ZIP CRC check establishes readable archive storage, not code safety. Source review remains required.

1.5.5 Task 5: Require a completed new process

Run the reviewed runner from the extracted directory, with a fresh working directory and explicit output path.

Self-check. The process is awaited with a timeout and checked return code. A process launch alone would not be evidence of completion.

1.5.6 Task 6: Compare aligned numeric outputs

Check exact ID sequence and status first. Then apply the frozen absolute-plus-relative numeric tolerance.

Self-check. Zero replay difference does not establish physical accuracy. The original workflow can be reproduced even when its scientific assumptions are wrong.

1.5.7 Task 7: Detect tampering on a copy

Append bytes to a copied request payload. Verify the reference copy is unchanged and the copied bundle is rejected.

Self-check. This change can preserve parsed values while changing bytes. A sender replacing both payload and manifest remains outside this integrity check.

1.5.8 Task 8: Create a defensible handover card

Keep passed, failed and untested claims separate. Adapt the public study prompts to your own project artifacts.

Self-check. Week 18 remains written and oral examination. This notebook neither publishes live exam answers nor introduces a new weekly graded task.

1.6 Evidence handover and summary

Package a bounded workflow, reproduce it in a fresh process, audit artifacts, and defend claim, evidence and limitation. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

1.7 Primary references

- NumPy reproducibility policy: https://numpy.org/doc/stable/reference/random/compatibility.html.
- scikit-learn model persistence: https://scikit-learn.org/stable/model_persistence.html.